

Cos'è iOS?:

Definizione:

Sistema operativo mobile sviluppato da Apple Inc, usato su iPhone e iPod Touch.

Varianti:

- iPadOS → per iPad.
- watchOS → per Apple Watch.
- tvOS → per Apple TV.

Basi tecniche:

Deriva da macOS, sistema operativo UNIX-like.

Condivide con macOS molte parti (kernel, librerie, framework).

Closed source (ma non del tutto):

iOS è closed source → codice non accessibile.

Tuttavia, molte componenti sono open-source:

1. Darwin

Sistema operativo base di macOS, iOS, watchOS, tvOS.

Include kernel XNU (open source, tranne i device driver).

Dal 2017 sono open anche le parti ARM.

2. Framework:

Alcuni framework inclusi in iOS sono open source:

WebKit, Foundation, Bonjour, FoundationDB, ecc.

3. Tool e linguaggi:

Swift e il runtime di Objective-C → open source.

Gran parte dei tool di sviluppo → open (LLVM, LLDB, Clang, ecc.).

Concetti e filosofia di iOS:

Sistema app-centric:

Filosofia: Ogni funzione è veicolata da un'app.

In iOS tutto è un'app.

Anche l'interfaccia utente principale (SpringBoard, la schermata Home) è un'applicazione.

Metafore di interazione:

Interazione diretta tramite tocco e gesti: *tap, swipe, pinch, pan*, ecc.

Non è mediata da dispositivi esterni (es. mouse) -> Favorisce immediatezza e intuitività.

Design:

Guidato dalle HIG (Human Interface Guidelines) di Apple.

Obiettivi:

- Interfacce intuitive e consistenti.
- Grande attenzione all'accessibilità (per utenti con disabilità).

HIG Obbligatorie per entrare nell'App Store

Privacy e sicurezza:

Distribuzione centralizzata del software → tutto passa dall'App Store.

Installazione di app provenienti da terze parti non ammessa!

Aggiornamenti di sistema forniti direttamente da Apple.

Supporto a:

- Crittografia.
- Autenticazione biometrica (Face ID, Touch ID).

Policy di raccolta dati minimale → Apple enfatizza protezione della privacy.

Recap:

iOS è un sistema app-centrico, pensato per interazione diretta e intuitiva.

Ha un design fortemente controllato da Apple (HIG).

La sicurezza e la privacy sono garantite da distribuzione centralizzata e strumenti crittografici.

App per dispositivi mobili:

Definizione:

Un'app è un pacchetto software che consente all'utente di eseguire un task specifico o una famiglia di task correlati.

Esempi: gestione della posta, prendere note, social network, ecc.

Contiene:

- eseguibili,
- metadati,
- risorse (icone, immagini, suoni),
- asset (contenuti vari).

GUI (Graphical User Interface):

Le app sono dotate di una GUI semplificata rispetto ai software desktop.

Le interfacce devono essere intuitive e leggere, adatte agli schermi piccoli e al touch.

Interazione con la sensoristica:

Le app sfruttano la sensoristica integrata nei dispositivi mobili:

- Touch screen.
- Accelerometro.
- Giroscopio.
- Fotocamera.
- Altri sensori (GPS, microfono, ecc.).

Questo le rende più "contestuali" rispetto ai programmi desktop.

Distribuzione:

Le app possono essere:

- Pre-installate sul dispositivo.
- Scaricate da store ufficiali.

Su iOS la distribuzione è (quasi) esclusivamente tramite App Store.

Architettura di un'app iOS:

Paradigma MVC (Model – View – Controller):

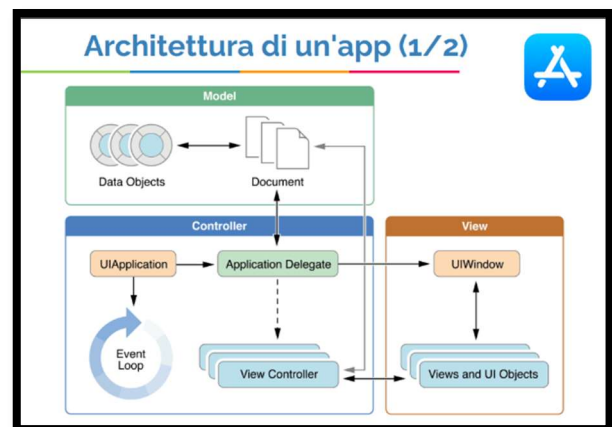
È il paradigma architetturale adottato da Apple per lo sviluppo delle app.

Ogni parte dell'app è separata in 3 componenti principali:

- Model → dati e logica.
- View → interfaccia grafica.
- Controller → gestisce l'interazione tra i due.

Flusso di esecuzione:

1. UIApplication → entry point dell'app, avvia il ciclo di eventi (Event Loop).
2. Application Delegate → gestisce eventi globali (avvio app, chiusura, notifiche push, ecc.).
3. View Controller → gestisce la logica di una parte dell'interfaccia.
4. UIView / UI Objects → gli oggetti visivi sullo schermo.



View:

Componente che gestisce l'interfaccia utente.

Una "view" è un rettangolo sullo schermo.

Possono essere organizzate in gerarchie.

Possono essere built-in (fornite dal sistema) o personalizzate dallo sviluppatore.

Controller:

Fa da intermediario tra Model e View.

Funzioni principali:

- Aggiornare le view quando cambia il model.
- Rispondere alle interazioni utente (touch, gesture).
- Coordinarsi con altri controller e oggetti per gestire task complessi.

Model:

Contiene i dati e la logica relativa.

Rappresenta gli oggetti di dominio e le loro relazioni.

Esempi:

- Stato dell'app.
- Accesso alla rete.
- Persistenza dati (database, file).

Recap:

- Model → logica e dati.
- View → interfaccia grafica.
- Controller → gestisce la comunicazione e le interazioni.

L'MVC rende le app modulari, mantenibili e facilmente estendibili.

Gestione degli eventi in iOS:

Event loop:

iOS segue un ciclo di eventi (event loop) → le app reagiscono agli eventi generati dall'utente (tap, swipe, pinch, ecc.).

È il cuore dell'interazione event-driven.

Flusso degli eventi:

1. Interazione utente: L'utente compie un gesto sul touch screen (tap, swipe, pinch).
2. Operating System: Il sistema operativo cattura l'evento hardware.
3. Port: L'evento viene incanalato verso un canale di comunicazione (port).
4. Event queue: Gli eventi vengono messi in una coda (in ordine di arrivo).
5. Main run loop: Ciclo principale dell'applicazione che legge la coda degli eventi.
6. Application object: Oggetto principale che riceve gli eventi e li smista.
7. Core objects: Gestiscono la logica e aggiornano la UI in base all'evento ricevuto.



Comunicazione bidirezionale:

Una volta elaborato, il risultato dell'evento (es. animazione, cambiamento UI) torna verso l'utente tramite output visivo o tattile.

Recap:

iOS è un sistema event-driven.

Gli eventi passano da input utente → OS → coda eventi → app → UI.

Il main run loop garantisce la gestione continua e ordinata degli eventi.

Ciclo di vita di un'app iOS:

Stati principali:

1. Not running: L'app non è stata avviata oppure è stata chiusa/terminata.
2. Inactive:

L'app è in foreground ma non riceve eventi.

Stato temporaneo, di breve durata (es. quando appare una chiamata in arrivo che blocca momentaneamente l'app).

3. Active:

L'app è in foreground e riceve eventi.

È lo stato "normale" durante l'uso da parte dell'utente.

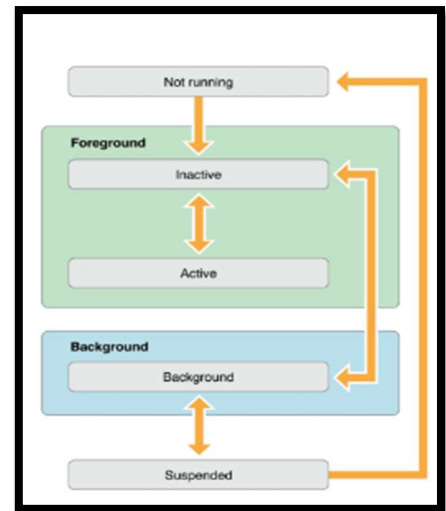
4. Background:

L'app è in background ma sta ancora eseguendo operazioni (es. riproduzione musicale, download, navigazione GPS).

5. Suspended:

L'app è in background ma non sta più eseguendo operazioni.

Rimane in memoria per poter essere riattivata velocemente.



Transizioni:

Not running → Foreground (Inactive → Active) → quando l'utente avvia l'app.

Active → Inactive → quando arriva un'interruzione (es. telefonata).

Inactive → Background → quando l'utente passa ad un'altra app.

Background → Suspended → quando l'app termina le operazioni in corso e viene "congelata".

Da Suspended → Foreground → può tornare attiva senza riavvio completo.

Recap:

Foreground: Inactive / Active.

Background: Background / Suspended.

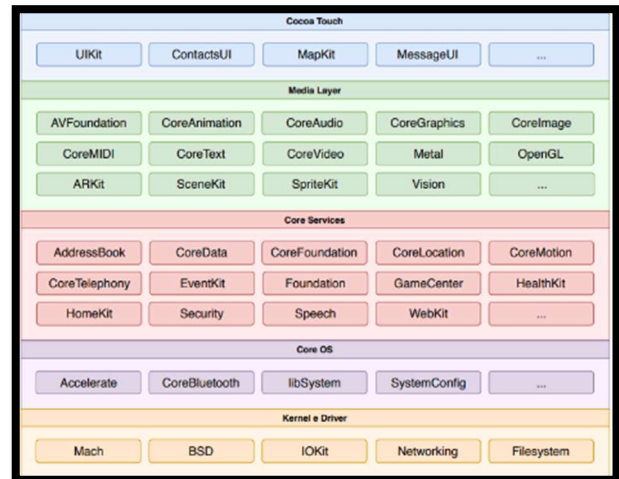
Not running: app chiusa o mai avviata.

Questo modello serve a risparmiare risorse (CPU, batteria, memoria) mantenendo comunque l'app pronta alla riattivazione.

Stack software di iOS:

L'architettura di iOS è divisa in 5 livelli principali:

1. Cocoa Touch → interfaccia e app.
2. Media Layer → grafica, audio, video.
3. Core Services → servizi di sistema per app.
4. Core OS → librerie e servizi di basso livello.
5. Kernel & Driver → interfaccia diretta con l'hardware.



Cocoa Touch:

Cos'è:

È il principale framework per sviluppare software su piattaforma iOS.

È orientato alle interfacce utente e all'interazione tramite tocco (touch screen).

Deriva da Cocoa (framework di macOS).

Scritto in Objective-C (oggi anche Swift).

Componenti principali:

UIKit:

Sub-framework fondamentale.

Fornisce API per la creazione di app native:

- Gestione interfaccia utente.
- Ciclo di vita delle app.
- Gestione eventi touch e gesture.

Altri sub-framework GUI:

- ContactsUI → accesso e gestione dei contatti.
- MapKit → mappe e geolocalizzazione.
- MessageUI → invio e composizione email.
- ... e molti altri.

Cocoa Touch = livello più alto dello stack software iOS.

Permette allo sviluppatore di costruire app native sfruttando API già pronte.

Fondamentale per: GUI, eventi touch, accesso ai servizi di sistema.

UIKit:

Definizione:

È l'infrastruttura architetturale ed implementativa per la creazione di app iOS.

Fornisce gli strumenti essenziali per sviluppare interfacce grafiche e interazioni utente.

UIKit fornisce:

1. UI elements (bottoni, campi di testo, finestre).
2. Event handling (gesture, notifiche).
3. Gestione sistema (ciclo di vita app, risorse, tastiera).
4. Supporto hardware (animazioni GPU, sensori).

UIKit = il cuore della programmazione di app iOS.

Media Layer:

Il Media Layer è responsabile della user experience visiva e sonora delle app iOS.

Immagini:

CoreImage → immagini e riconoscimento.

Audio/Video:

AVFoundation → registrazione e playback multimediale.

CoreAudio/CoreVideo → gestione a basso livello di stream audio/video.

Rendering 2D/3D:

CoreGraphics/CoreAnimation → rendering e animazioni 2D.

OpenGL/Metal → grafica 3D.

Core Services:

Il livello Core Services fornisce le API essenziali che permettono alle app di funzionare oltre la UI.

API essenziali:

1. Foundation → standard library.
2. CoreData → persistenza dati (tipo ORM).
3. CoreLocation → GPS e orientamento.
4. CoreMotion → sensori di movimento.
5. WebKit → browser e rendering web.

Core OS:

Il livello Core OS è ciò che collega direttamente le app e i servizi al kernel e all'hardware.

Definizione:

È il livello più basso dello userland.

Gestisce la comunicazione con i device driver e il kernel.

Fornisce API fondamentali per le funzioni di sistema.

Componenti principali:

1. libSystem → librerie di base C e syscall.
2. Accelerate → calcolo numerico e DSP ottimizzati.
3. CoreBluetooth → gestione BLE.
4. SystemConfiguration → configurazione e stato rete.

Darwin:

Cos'è:

È il sistema operativo di base su cui si fondano iOS e macOS.

Non è un kernel, ma un sistema completo che include:

- Kernel.
- Librerie.
- Servizi di basso livello.

Caratteristiche:

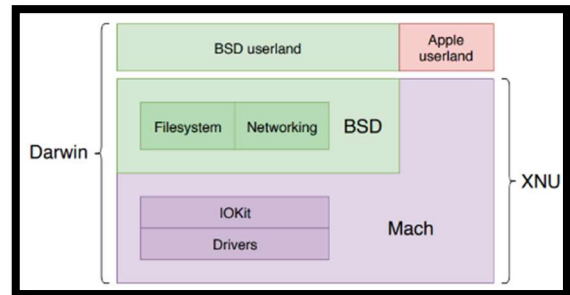
Open source, rilasciato con licenza APSL (*Apple Public Source License*).

Kernel: XNU (acronimo di *X is Not Unix*).

Combina Mach (microkernel) e BSD (userland Unix modificato).

Architettura:

1. Mach: Gestione processi, memoria, driver (tramite IOKit).
2. BSD: Filesystem, networking, API POSIX. (Fornisce l'ambiente Unix)
3. Userland BSD: Strumenti e comandi tipici di Unix.
4. Apple userland: Componenti proprietari Apple sopra Darwin (es. Cocoa, framework iOS).



Recap:

Darwin = base open source dei sistemi Apple.

Include: Mach + BSD + IOKit + Apple userland.

Kernel = XNU (Mach + BSD).

iOS/macOS hanno fondamenta Unix-like ma con personalizzazioni Apple.

XNU:

Definizione:

Acronimo di "X is Not Unix".

Deriva da Mach (microkernel) + BSD (monolitico).

È un kernel ibrido.

Perché ibrido?

1. Kernel monolitico: Tutte le funzionalità core in *kernel space*.

Pro: Alte prestazioni, controllo diretto dell'hardware.

Contro: Difficile da mantenere, meno sicuro, bug più gravi.

2. Microkernel (Mach): Funzionalità minime nel kernel, il resto in *user space* come servizi.

Pro: Più sicuro, modulare, semplice.

Contro: Performance peggiori (tanti context switch, IPC più costosi).

3. Kernel ibrido (XNU): Unisce i due modelli:

Da Mach eredita modularità e separazione.

Da BSD eredita parti critiche ad alte prestazioni (filesystem, networking, gestione processi) implementate in *kernel space*.

Recap:

Mach = microkernel → modularità, sicurezza.

BSD = monolitico → performance e servizi avanzati.

XNU = ibrido → il meglio dei due mondi (È il cuore di Darwin e quindi di iOS/macOS).

Mach e BSD:

Mach: fornisce la struttura di base del kernel: scheduling, IPC, gestione memoria, thread.

Funzionalità principali:

1. CPU scheduling → scheduler preemptive basato su code a priorità.
2. IPC (Inter Process Communication) → messaggi scambiati tra processi.
3. RPC (Remote Procedure Call) → implementato su Mach IPC.
4. Supporto al soft real-time.
5. Memoria virtuale → paginata.
6. Thread lato kernel.

BSD: fornisce i servizi avanzati di sistema: filesystem, rete, sicurezza, API Unix.

Funzionalità principali:

1. Protezione memoria → mappatura e protezione delle pagine.
2. VFS (Virtual File System) → gestione filesystem:
3. APFS, HFS/HFS+, NTFS (sola lettura), FAT32/EXFAT, ecc.
4. Networking → stack TCP/IP, socket, firewall.
5. Modello di sicurezza UNIX → utenti, gruppi, permessi, policy.
6. API POSIX e syscall → compatibilità Unix.
7. Modello processi → process ID, proprietari, gruppi, segnali.
8. Thread lato utente (pthread) → basati su thread Mach.

Recap:

Mach → cuore del kernel: scheduling, IPC, memoria, thread.

BSD → funzionalità di sistema: processi, sicurezza, file system, rete.

Insieme → formano il kernel XNU (ibrido).

IOKit:

Permette ad Apple di gestire driver in modo modulare, riusabile e orientato agli oggetti.

Definizione:

Framework orientato agli oggetti (C++).

Serve per lo sviluppo di device driver.

È privato su iOS (non accessibile agli sviluppatori di app), ma usato internamente dal sistema operativo.

Concetti principali:

In IOKit, ogni dispositivo/servizio è rappresentato come una classe:

1. Family:

Modella categorie di dispositivi (es. USB, HID, disco, rete).

Permette il riuso di codice tra driver di dispositivi simili.

Es: tutti i driver di dispositivi USB condividono parte della logica.

2. Driver:

Oggetto che gestisce una specifica tipologia di dispositivo.

Rappresenta il dispositivo in modo astratto.

Ogni driver può appartenere a una o più *family*.

3. Nub:

Rappresenta un canale di comunicazione con il dispositivo.

Astrazione di:

- Bus.
- Buffer.
- Area di memoria mappata per I/O.

Ogni driver espone uno o più nub, a seconda dei dispositivi che sa controllare.

Recap:

IOKit = framework per driver, basato su classi C++ (Interno => non disponibile alle app).

Componenti:

1. Family → categorie di dispositivi.
2. Driver → rappresentazione astratta di un dispositivo.
3. Nub → canale di comunicazione col dispositivo.

APFS – Apple File System:

È il filesystem di default su iOS, macOS e watchOS/tvOS.

Caratteristiche principali:

1. Inode a 64 bit

Permette di indirizzare fino a $\sim 10^{19}$ file → enorme scalabilità.

2. CoW (Copy-On-Write)

Quando un file viene copiato, non viene duplicata la memoria subito.

Viene duplicata solo se viene modificato.

Riduce lo spazio occupato.

Rende superfluo il journaling, perché le modifiche avvengono su nuove aree di memoria.

3. Snapshots

“Foto” istantanee dello stato di file/volumi.

Permettono il ripristino rapido di un volume a uno stato precedente.

4. Partizioni dinamiche

Supporto a dimensione variabile: più partizioni possono condividere lo stesso spazio, invece di avere container fissi.

5. Crittografia avanzata

Supporto integrato a cifratura per file.

Modalità:

- Chiave singola → univoca per file.
- Chiave multipla → diverse parti del file cifrate con chiavi differenti.

Recap:

APFS è moderno, sicuro e scalabile.

Caratteristiche chiave:

1. Copy-on-Write → efficienza e sicurezza.
2. Snapshots → ripristino veloce.
3. Partizioni dinamiche → gestione flessibile dello spazio.
4. Crittografia integrata → sicurezza dei dati.

Secure Boot Chain:

La Secure Boot Chain garantisce che ogni componente del sistema operativo iOS sia autentico e firmato da Apple, impedendo modifiche non autorizzate.

Concetto base:

L'avvio procede per stadi sequenziali.

Ogni stadio viene verificato crittograficamente da quello precedente → boot chain of trust.

Garantisce che il sistema non sia stato modificato da terze parti (anti-manomissione / anti-jailbreak).

Memorie coinvolte:

1. SecureROM

Memoria read-only, scritta in fabbrica da Apple (Inaccessibile al kernel).

Contiene BootROM (prime istruzioni) + root CA key usata per verificare le firme digitali degli stadi successivi.

2. NAND Flash

Memoria riscrivibile divisa in partizioni fisiche:

- BOOT → contiene LLB e iBoot, non accessibile al kernel, può essere sovrascritta da aggiornamenti.
- FSYS → contiene il filesystem (kernel + userspace).

Stadi della Boot Chain:

1. BootROM (SecureROM):

Primo codice eseguito.

Verifica e lancia il successivo LLB.

2. LLB (Low Level Bootloader) *(opzionale, solo in vecchi modelli):*

Presente su iPhone 6 e precedenti.

Esegue subroutine di setup e verifica l'integrità di iBoot.

3. iBoot:

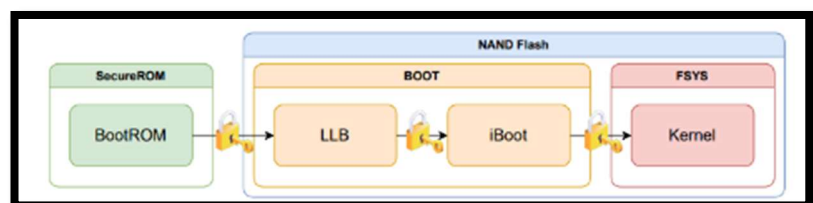
Bootloader principale.

Verifica integrità del kernel XNU.

Carica in memoria e avvia il sistema.

4. Kernel:

Ultimo stadio: avvio del sistema operativo iOS.



Failures:

Se uno stadio fallisce, il dispositivo entra in:

- DFU (Device Firmware Update) mode → ripristino completo.
- Recovery mode → ripristino più leggero.

Dipende da dove fallisce la verifica (BootROM → LLB → iBoot → Kernel).

Feature di sicurezza in iOS:

iOS applica sicurezza a più livelli:

- Boot (chain of trust),
- Kernel (integrità + protezione memoria),
- Dati (cifatura pervasiva con SEP),
- Applicazioni (firma + sandbox + permessi).

Sicurezza del sistema:

1. Secure boot chain: garantisce che il sistema operativo non sia stato manomesso.
2. Aggiornamenti software: verificati dai server Apple e firmati digitalmente con chiavi univoche.
3. SEP (Secure Enclave Processor): coprocessore dedicato per dati sensibili (biometrici, passcode):
 - Non accessibile direttamente al kernel.
 - Interagisce solo tramite una message box sicura.

Sicurezza del kernel:

1. W^X (Write XOR Execute): una pagina di memoria può essere o scrivibile o eseguibile, mai entrambe → riduce exploit.
2. KASLR (Kernel Address Space Layout Randomization): il kernel viene caricato in posizioni di memoria casuali ad ogni avvio → ostacola attacchi.
3. KIP (Kernel Integrity Protection): impedisce modifiche alla memoria del kernel dopo il boot.
4. PAC (Pointer Authentication): autenticazione crittografica dei puntatori in memoria → previene attacchi su buffer overflow e ROP.

Sicurezza dei dati:

1. Coprocessore AES256: crittografia hardware dei dati (se presente).
2. File encryption: ogni file cifrato con chiavi uniche per file, derivate da passcode + SEP.
3. Supporto a partizioni dinamiche e snapshot per recovery sicura.

Sicurezza delle applicazioni:

1. Code signing: tutte le app devono essere firmate da Apple → verificate tramite AMFI (Apple Mobile File Integrity).
2. Sandboxing: ogni app gira in un ambiente isolato → niente accesso arbitrario a file/risorse di altre app.
3. Entitlement: accesso ad API sensibili (fotocamera, contatti, GPS) con consenso esplicito dell'utente.